CLOUD SECURITY & IAM

Shared Responsibility, Storage Policies and Access Control

Submitted by: Sujal Chouhan
Subject: Cloud Computing / Cloud Security

This assignment uses AWS as the example cloud platform. The same principles apply to Azure and Google Cloud, although their role names and policy syntax differ.

Safety note: Cloud trials may require identity or payment details and may create charges. Review current free-tier terms, set a budget alert where available, and do not share passwords, access keys, or private key files.

Task 1: Shared Responsibility Model for a Food Delivery App

For a Zomato-like app running on AWS, Azure or Google Cloud, the provider secures the cloud infrastructure, while the customer is responsible for securely configuring and using the services. Exact boundaries vary by service (virtual machines, managed databases, or serverless services).

Security area	Cloud provider responsibility	Customer / app-team responsibility
Physical facilities	Data-centre buildings, physical access, power and hardware protection.	Choose appropriate regions and service types; assess compliance needs.
Core cloud infrastructure	Secure physical servers, network backbone, and virtualization layer.	Configure cloud networks, subnets, routing, firewall/security-group rules and private access.
Identity and access	Provide IAM tools and secure identity services.	Use least privilege, MFA, strong authentication, review permissions and remove unused accounts.
App and operating system	For managed services, maintain the underlying platform; for VMs, responsibility is more limited.	Secure application code and APIs; patch guest OS and runtime when responsible; scan dependencies.
Data and privacy	Provide encryption features, key-management options and durable storage controls.	Classify customer/order/address data, configure access policies, manage keys where applicable, backup and retention.
Monitoring and incident response	Maintain provider-side service monitoring and publish security logs/alerts for supported services.	Enable and review audit logs, set alerts, investigate app activity, respond to exposed credentials and breaches.

Key idea: The provider secures the cloud OF the service; the customer secures what they put IN the cloud and how they configure and access it.

Task 2: Create a Limited-Access User (AWS example)

1. Sign in to the AWS Console using your authorized account. Avoid using the root user for routine administration.

2. Open **IAM** → **Users** → **Create user** and choose a descriptive name such as **fest-image-uploader**.

3. Grant only the permissions required for the task. Prefer a specific custom policy attached to this user or, better for teams, to a group. Do not attach AdministratorAccess or billing permissions.

4. If console sign-in is needed, configure it according to your lab requirements and require MFA where available. If programmatic access is needed, prefer short-lived credentials/roles where possible.

5. Test with the user: uploading to the assigned bucket should work, while unrelated services, billing pages and administration actions should be unavailable.

6. Review the permissions after the lab and remove the user or access that is no longer needed.

Note: AWS console labels and free-tier offers can change. Do not create resources you do not need, and check the billing dashboard/budget controls.

Task 3: Allow Uploads but Deny File Deletion

Example AWS IAM policy for a dedicated S3 bucket. Replace **YOUR-BUCKET-NAME** with your own bucket name. This policy allows uploading objects into the bucket and explicitly denies deleting objects. It does not grant broad access to other AWS services.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowUploadToOneBucket",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    },
    {
      "Sid": "DenyDeletingObjects",
      "Effect": "Deny",
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    }
  ]
}
```

Apply and test:

1. Open **IAM** → **Policies** → **Create policy** → **JSON**, paste the policy and replace the bucket placeholder.

2. Validate the policy, give it a name such as **UploadOnlyNoDelete**, create it, and attach it only to the intended user/group.

3. Ensure the bucket exists and the user has no other attached policy that grants extra permissions. Explicit Deny overrides an Allow for the same action.

4. Test uploading a harmless test image. Then attempt a delete in the lab: deletion should be denied. If listing bucket contents is also required, grant the minimum additional `s3:ListBucket` permission on the bucket ARN separately.

Important: This example blocks object deletion but does not by itself block every possible way data could be changed or permissions misconfigured. Keep the bucket private, enable Block Public Access, and test with a non-production bucket.

Task 4: IAM Roles for a Cricket Ticket Booking App

Use separate identities and least-privilege permissions rather than sharing one administrator login.

Role / user	Permissions	Example AWS implementation
Booking Viewer	Can view booking records but cannot create, edit or delete them.	A custom policy allowing only required read actions for the booking database/table or application API.
Booking Editor	Can create and update bookings; delete only if the business process explicitly requires it.	A custom policy for the required write/update actions on the specific booking resource. Avoid account-wide permissions.
User Account Manager	Can manage application user accounts, but should not automatically gain access to booking data or billing.	Use a narrowly scoped identity-management role or application-admin permission set. Restrict it to user-management actions only.

Implementation steps: Create three groups or role-based permission sets; create one separate test user for each; attach the matching policy; avoid granting billing/admin access; then test both allowed and denied actions. Exact policy actions depend on whether booking data lives in DynamoDB, RDS, or behind an application API.

Task 5: Real Security Incident — Football Australia (2024)

The Cloud Security Alliance reported in June 2025 that researchers found plaintext AWS access keys embedded in Football Australia's website source. Those keys provided access to 127 digital storage containers; at least one S3 bucket was publicly accessible and contained personal information about football players, attendee purchase information, technical designs and source code. The report describes this as a combination of misconfiguration, excessive exposure and insecure software development.

How the Shared Responsibility Model could have helped (3–4 lines):

AWS provides controls such as IAM, S3 Block Public Access, logging and security monitoring, but the customer must configure and use them securely. Football Australia could have kept buckets private, used narrowly scoped and short-lived credentials instead of embedding long-term keys in website code, and reviewed access regularly. Automated configuration scans and alerts could have detected public storage earlier. This illustrates that using a cloud provider does not remove the customer's responsibility for data permissions and application secrets.

Source: Cloud Security Alliance, “The 2024 Football Australia Data Breach: A Case of Misconfiguration and Inadequate Change Control,” published 9 June 2025.

https://cloudsecurityalliance.org/blog/2025/06/09/the-2024-football-australia-data-breach-a-case-of-misconfiguration-and-inadequate-change-control

Conclusion

Cloud security is a shared responsibility. Providers protect the underlying cloud infrastructure, while customers must control identities, data access, application security, configurations and monitoring. Least privilege, explicit policy testing, private storage defaults and regular access reviews reduce the chance of accidental exposure.